UNIVERSIDAD SIGLO 21
La Educación Evoluciona

SEMINARIO DE PRÁCTICA DE INFORMÁTICA

Actividad Práctica 4 - Proyecto Integrador

SITRAC-MET

Sistema de gestión y trazabilidad de instrumentos de medición y calibraciones

Alumno	Ricardo Vidal
Universidad	Universidad Siglo 21
Entrega	Actividad Práctica 4
Tecnologías	Java 21 · Apache NetBeans · MySQL · JDBC
Repositorio	https://github.com/rickynorby-dot/VIDAL-RICARDO-AP1
Fecha	Noviembre de 2026

Índice

1. Introducción y continuidad incremental
2. Alcance de la versión integradora
3. Selección del patrón de diseño y justificación
4. Arquitectura y organización del proyecto Java
5. Persistencia de datos con MySQL y JDBC
6. Clases abstractas, interfaces y reutilización
7. Uso complementario de arreglos y ArrayList
8. Manejo de excepciones
9. Utilización de archivos
10. Presentación y verificación del desarrollo
11. Matriz de cumplimiento de la consigna
12. Repositorio GitHub y video de presentación
13. Conclusiones
14. Referencias bibliográficas

1. Introducción y continuidad incremental

SITRAC-MET es un proyecto orientado a centralizar la gestión y trazabilidad de instrumentos de medición utilizados en una organización industrial. El problema analizado inicialmente se relaciona con la utilización de planillas y registros dispersos para controlar la disponibilidad de los instrumentos, sus responsables, las calibraciones y las fechas de vencimiento. Esta forma de trabajo dificulta conocer el estado real de cada instrumento y aumenta el riesgo de utilizar equipos cuya calibración ya no se encuentra vigente.

La cuarta actividad retoma las decisiones desarrolladas en las entregas anteriores y las integra en una versión operacional más completa. En AP1 se definieron el problema, el alcance, los requerimientos y los casos de uso. En AP2 se avanzó con PUD, UML, el modelo relacional y la base MySQL. En AP3 se implementó el dominio mediante programación orientada a objetos, incorporando abstracción, herencia, encapsulamiento, polimorfismo, excepciones, estructuras de control, búsqueda y ordenación. En esta instancia se consolida la arquitectura, se aplica un patrón de diseño y se integra la persistencia mediante JDBC.

El resultado es un prototipo que mantiene el alcance académico definido: no intenta reemplazar una plataforma metrológica comercial, sino demostrar una solución incremental y coherente con los requerimientos del negocio. Esta decisión se corresponde con el concepto de prototipo operacional planteado por Kendall y Kendall (2011), en el que los módulos implementados forman parte de la evolución hacia el sistema final.

2. Alcance de la versión integradora

La versión AP4 incorpora las funcionalidades necesarias para demostrar la integración entre el modelo orientado a objetos, la interfaz de consola, las reglas del dominio, la base de datos y mecanismos complementarios de almacenamiento. El alcance final del prototipo comprende:

- Registro de instrumentos eléctricos y mecánicos.
- Consulta y búsqueda de instrumentos por código.
- Control del estado operativo y de la vigencia de calibración.
- Registro de calibraciones y actualización de estado.
- Asignación y devolución de instrumentos.
- Consulta de datos persistidos en MySQL.
- Registro de acciones relevantes en un archivo de auditoría.
- Manejo controlado de errores de negocio y de persistencia.

La entrega conserva además las correcciones incorporadas durante el desarrollo previo: diagramas con mejor legibilidad, separación entre atributos de calidad y restricciones tecnológicas, encapsulamiento de la conexión a base de datos y eliminación de credenciales fijas dentro de la lógica del programa.

3. Selección del patrón de diseño y justificación

3.1. Patrón seleccionado: Modelo-Vista-Controlador (MVC)

Se seleccionó el patrón Modelo-Vista-Controlador (MVC). La elección no responde solamente a un requisito académico: resulta adecuada porque SITRAC-MET necesita separar tres responsabilidades que evolucionan de manera diferente. La presentación puede cambiar sin modificar las reglas de negocio; la lógica de asignación y calibración debe mantenerse independiente de la interfaz; y la persistencia puede evolucionar sin trasladar SQL a las clases que interactúan con el usuario.

El Módulo 4 plantea MVC como una alternativa para organizar una aplicación en componentes independientes y facilitar su gestión, mantenimiento y escalabilidad. En esta implementación la Vista no ejecuta SQL ni decide reglas del dominio. El Controlador interpreta la opción seleccionada y coordina las operaciones. El Modelo conserva los objetos y reglas del negocio, mientras que los DAO encapsulan la interacción con MySQL.

Componente	Implementación	Responsabilidad
Vista	ConsolaVista	Presenta el menú, solicita datos y muestra resultados o errores.
Controlador	SitracMetControlador	Coordina los casos de uso y relaciona Vista, Modelo, persistencia y auditoría.
Modelo	InstrumentoMedicion, subclases, Asignacion, Calibracion, GestorInstrumentos	Representa el dominio y aplica reglas de disponibilidad, calibración, asignación y devolución.
Persistencia	RepositorioInstrumentos, InstrumentoDAO, CalibracionDAO, AsignacionDAO	Ejecuta las operaciones JDBC contra MySQL sin mezclar SQL con la interfaz.

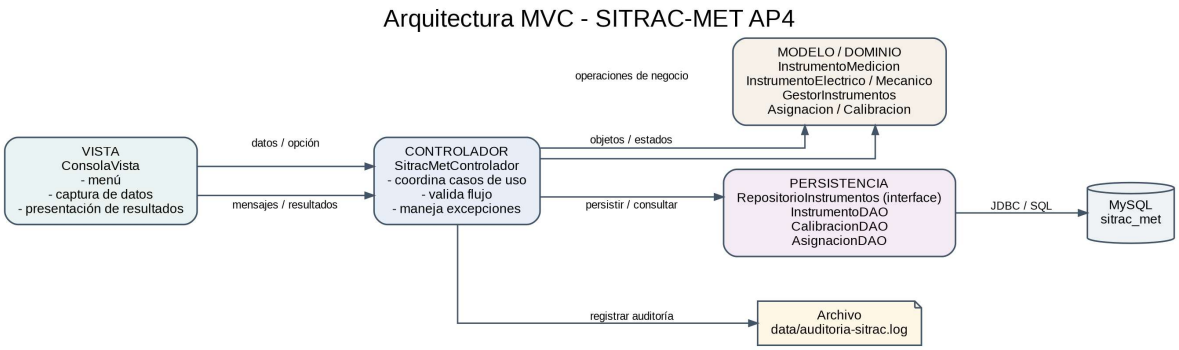


Figura 1. Arquitectura MVC implementada en SITRAC-MET AP4.

4. Arquitectura y organización del proyecto Java

El proyecto se organizó en paquetes con responsabilidades específicas. Esta estructura mantiene la coherencia con el diseño orientado a objetos de AP3 y, al mismo tiempo, incorpora las capas necesarias para aplicar MVC y JDBC.

Paquete	Contenido principal
sitracmet.app	Punto de entrada SitracMetApp.
sitracmet.vista	ConsolaVista.
sitracmet.controlador	SitracMetControlador.
sitracmet.modelo	Clases del dominio, estados y resultados.
sitracmet.servicio	GestorInstrumentos y reglas de negocio.
sitracmet.dao	ConexionBD, repositorio y DAO JDBC.
sitracmet.dto	Datos recuperados desde MySQL para presentación.
sitracmet.excepcion	Excepciones propias del dominio y PersistenciaException.
sitracmet.util	AuditoriaArchivo.

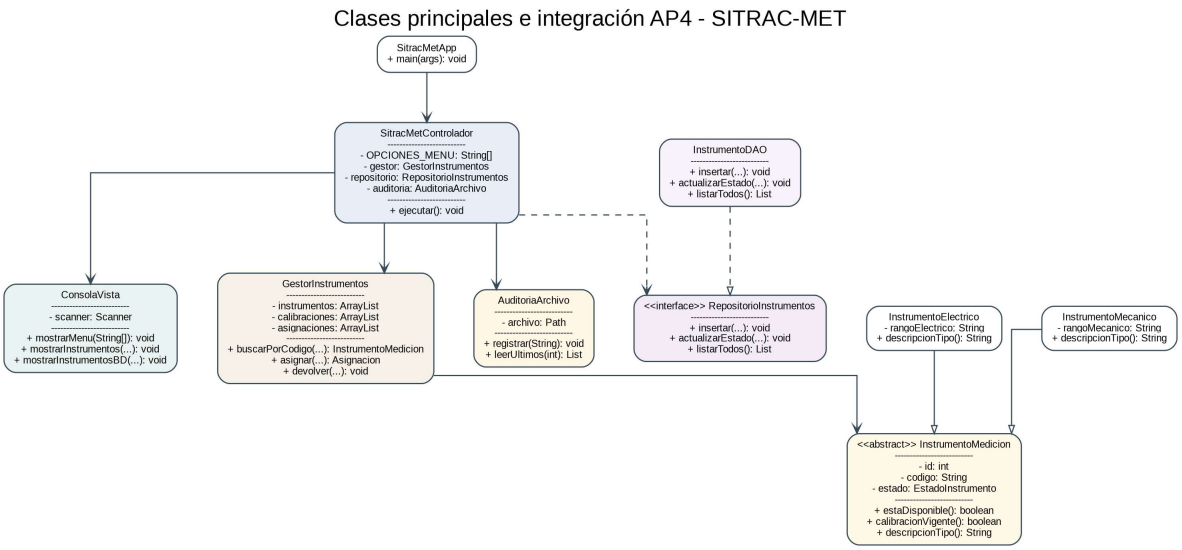


Figura 2. Clases principales y relaciones incorporadas en la integración AP4.

4.1. Menú y control del flujo

El programa mantiene una ejecución iterativa mediante un ciclo while. El menú se representa con un arreglo de opciones de tamaño fijo y el controlador utiliza switch para derivar cada selección al método correspondiente. De esta manera, la Vista solo captura y presenta información, mientras que el Controlador decide qué caso de uso ejecutar.

```
private static final String[] OPCIONES_MENU = {
    "1. Registrar instrumento eléctrico",
    "2. Registrar instrumento mecánico",
    "3. Listar instrumentos en memoria",
    ...
    "9. Consultar instrumentos persistidos en MySQL",
    "10. Ver últimas acciones de auditoría",
    "0. Salir"
};
```

5. Persistencia de datos con MySQL y JDBC

La persistencia continúa utilizando la base de datos MySQL definida en AP2. Para evitar que la lógica SQL quede acoplada al controlador, las operaciones se agrupan en clases DAO. La conexión se obtiene mediante ConexionBD y utiliza variables de entorno para URL, usuario y contraseña. Esto evita almacenar credenciales reales directamente en el código fuente.

```
String url = System.getenv().getOrDefault(
    "SITRAC_DB_URL",
    "jdbc:mysql://localhost:3306/sitrac_met?..."
);
return DriverManager.getConnection(url, usuario, clave);
```

5.1. Operaciones implementadas

Operación	Clase	Implementación
Conexión	ConexionBD	Abre una Connection JDBC utilizando la configuración del entorno.
Insertar	InstrumentoDAO / CalibracionDAO / AsignacionDAO	Utiliza INSERT con PreparedStatement.
Consultar	InstrumentoDAO.listarTodos()	Ejecuta SELECT con JOIN y recupera resultados mediante ResultSet.
Actualizar	InstrumentoDAO.actualizarEstado()	Actualiza el estado del instrumento luego de asignar, devolver o calibrar.

Operación	Clase	Implementación
Cerrar asignación	AsignacionDAO.cerrarAsignacionAbierta()	Registra fecha y condición de devolución.

Se utiliza PreparedStatement para parametrizar los valores y try-with-resources para garantizar el cierre de Connection, PreparedStatement y ResultSet. La consulta de instrumentos combina las tablas instrumento, tipo_instrumento y calibracion para mostrar desde la interfaz el código, tipo, marca, modelo, estado y próximo vencimiento.

```
try (Connection cn = ConexionBD.abrir();
     PreparedStatement ps = cn.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        resultado.add(new InstrumentoPersistido(...));
    }
} catch (SQLException e) {
    throw new PersistenciaException("No fue posible consultar...", e);
}
```

5.2. Integridad entre dominio y persistencia

Las modificaciones de estado se realizan primero mediante las reglas del dominio y luego se reflejan en MySQL. Por ejemplo, una asignación solo puede concretarse si el instrumento está disponible y su calibración se encuentra vigente. Después de superar esas validaciones, se registra la asignación y se actualiza el estado persistido. Esta secuencia evita que la base de datos registre como válida una operación que el modelo de negocio rechaza.

6. Clases abstractas, interfaces y reutilización

La solución mantiene la clase abstracta InstrumentoMedicion definida en AP3. Esta clase reúne el estado y comportamiento común a todos los instrumentos y obliga a las subclases a implementar descripcionTipo(). InstrumentoElectrico e InstrumentoMecanico especializan ese comportamiento. Al almacenar ambas clases en una colección de InstrumentoMedicion, el programa utiliza polimorfismo para invocar la implementación correcta sin preguntar el tipo concreto del objeto.

```
public abstract class InstrumentoMedicion {
    public abstract String descripcionTipo();
}

public class InstrumentoElectrico extends InstrumentoMedicion {
    @Override
    public String descripcionTipo() {
        return "Instrumento eléctrico [" + rangoElectrico + "]";
    }
}
```

Además, AP4 incorpora la interfaz RepositorioInstrumentos. El controlador depende de esa abstracción y no de los detalles internos de InstrumentoDAO. Esto permite reemplazar la implementación de persistencia sin alterar el flujo principal y reduce el acoplamiento entre componentes.

```
public interface RepositorioInstrumentos {
    void insertar(InstrumentoMedicion instrumento, int idTipo, int idSector)
        throws PersistenciaException;
    void actualizarEstado(InstrumentoMedicion instrumento)
        throws PersistenciaException;
    List<InstrumentoPersistido> listarTodos()
        throws PersistenciaException;
}
```

7. Uso complementario de arreglos y ArrayList

La consigna solicita utilizar arreglos y ArrayList de manera complementaria. Para que la elección no sea artificial, cada estructura se aplica donde sus características resultan apropiadas.

Estructura	Uso	Justificación
String[]	Opciones del menú principal.	La cantidad de opciones es conocida y permanece fija durante la ejecución.
ArrayList<InstrumentoMedicion>	Instrumentos cargados en el gestor.	La cantidad de instrumentos crece dinámicamente.
ArrayList<Calibracion>	Historial de calibraciones.	Se incorporan registros durante la ejecución.
ArrayList<Asignacion>	Asignaciones y devoluciones.	La colección necesita altas y recorridos dinámicos.

El ArrayList también se utiliza para aplicar la búsqueda secuencial y el ordenamiento por inserción desarrollados en AP3. Por lo tanto, la cuarta entrega no elimina los algoritmos anteriores, sino que los conserva dentro del modelo integrado.

8. Manejo de excepciones

El tratamiento de excepciones se aplica en dos niveles. En el dominio se mantienen excepciones específicas que expresan problemas del negocio; en la persistencia, las SQLException se traducen a PersistenciaException para evitar que detalles técnicos de JDBC se propaguen hasta la Vista.

Excepción	Situación controlada
CodigoDuplicadoException	Se intenta registrar un código de instrumento ya existente en memoria.
InstrumentoNoEncontradoException	No existe el código solicitado.
InstrumentoNoDisponibleException	Se intenta asignar o devolver un instrumento en un estado incompatible.
CalibracionVencidaException	Se intenta asignar un instrumento cuya calibración no está vigente.
PersistenciaException	Falla de conexión, consulta, inserción o actualización MySQL.
NumberFormatException / DateTimeParseException	El usuario ingresa un número o una fecha con formato incorrecto.
IOException	No puede escribirse o leerse el archivo de auditoría.

El controlador concentra los bloques catch y presenta mensajes comprensibles. De esta forma, una excepción no finaliza abruptamente el programa y el usuario puede continuar operando.

9. Utilización de archivos

Aunque el uso de archivos es opcional en la consigna, se incorporó una función simple de auditoría para practicar el almacenamiento secuencial en Java sin reemplazar la base de datos. El archivo no se utiliza como persistencia principal; su función es conservar un registro de acciones relevantes realizadas durante la ejecución.

```
public void registrar(String accion) throws IOException {
    Files.createDirectories(archivo.getParent());
    Files.writeString(archivo, linea, StandardCharsets.UTF_8,
        StandardOpenOption.CREATE, StandardOpenOption.APPEND);
}
```

Las altas, calibraciones, asignaciones y devoluciones generan una línea con fecha y hora. La opción 10 del menú permite recuperar las últimas acciones mediante leerUltimos(). Esto complementa MySQL con un mecanismo de archivo cuya finalidad es distinta y claramente delimitada.

10. Presentación y verificación del desarrollo

El proyecto fue compilado con JDK 21. Para verificar el comportamiento del modelo, el controlador, las colecciones y el archivo se incorporó un modo de demostración (--demo), que deshabilita únicamente las operaciones JDBC cuando no existe un servidor MySQL disponible. El modo normal utiliza la misma aplicación y ejecuta los DAO contra la base sitrac_met.

Evidencia de ejecución - menú y ArrayList

```
SITRAC-MET - MENÚ PRINCIPAL
1. Registrar instrumento eléctrico
2. Registrar instrumento mecánico
3. Listar instrumentos en memoria
4. Buscar instrumento por código
5. Registrar calibración
6. Asignar instrumento
7. Registrar devolución
8. Consultar asignaciones
9. Consultar instrumentos persistidos en MySQL
10. Ver últimas acciones de auditoría
0. Salir

Seleccione una opción: 3
Código | Tipo | Marca/Modelo | Estado | Calibración
INS-001 | Instrumento eléctrico [0-1000 V] | Fluke 179 | DISPONIBLE | venc.: 2027-05-06
INS-002 | Instrumento mecánico [0-10 bar] | Wika 232.50 | DISPONIBLE | venc.: 2026-11-05
```

Figura 3. Ejecución del menú y listado polimórfico sobre ArrayList.

Evidencia de ejecución - controlador, reglas y archivo

```
Seleccione una opción: 6
Código de instrumento: INS-001
Condición de entrega: Equipo en buenas condiciones
Asignación registrada con id 1

Seleccione una opción: 8
#1 | INS-001 | Juan Pérez | ABIERTA

Seleccione una opción: 7
Código de instrumento: INS-001
Condición de devolución: Sin novedades
Devolución registrada.

Seleccione una opción: 10
Últimas acciones de auditoría:
2026-11-06 23:37:45 | Asignación INS-001 a L-1001
2026-11-06 23:37:45 | Devolución INS-001
```

Figura 4. Asignación, devolución y recuperación del archivo de auditoría.

La evidencia muestra que el instrumento INS-001 puede asignarse porque está disponible y posee calibración vigente. Luego de la devolución la asignación pasa a estado cerrada y las acciones quedan registradas en el archivo de auditoría. La validación de MySQL se realiza ejecutando los scripts incluidos en la carpeta sql y utilizando la opción 9 del menú en modo normal.

10.1. Configuración del entorno

Elemento	Configuración
IDE	Apache NetBeans (proyecto Maven).
Java	JDK 21.
Base de datos	MySQL, esquema sitrac_met.
Driver	MySQL Connector/J, declarado en pom.xml.
Repositorio	GitHub.

11. Matriz de cumplimiento de la consigna

Requisito de AP4	Evidencia en SITRAC-MET
Correcciones de entregas anteriores	Se conserva la arquitectura mejorada, diagramas legibles y separación de configuración/credenciales.

Requisito de AP4	Evidencia en SITRAC-MET
Patrón de diseño y justificación	MVC: ConsolaVista, SitracMetControlador y Modelo/servicio, con persistencia desacoplada.
Persistencia MySQL	ConexionBD + DAO JDBC + scripts del esquema y datos.
Consultas y actualizaciones	InstrumentoDAO.listarTodos(), actualizarEstado(), inserciones y cierre de asignaciones.
Resultados en interfaz	ConsolaVista.mostrarInstrumentosBD().
Excepciones MySQL	SQLException se encapsula en PersistenciaException y se trata en el controlador.
Clase abstracta o interfaz	InstrumentoMedicion (abstracta) y RepositorioInstrumentos (interface).
Arreglos	String[] OPCIONES_MENU.
ArrayList	Instrumentos, calibraciones y asignaciones en GestorInstrumentos.
Archivos	AuditoriaArchivo para registrar y recuperar acciones.
Proyecto completo	Código, SQL, diagramas, evidencias, README e informe en el paquete AP4.
Video ~3 minutos	Se adjunta guion orientativo con recorrido del problema, arquitectura, POO, MySQL y cierre.

12. Repositorio GitHub y video de presentación

La versión completa del proyecto se mantiene en GitHub como repositorio académico incremental:

https://github.com/rickynorby-dot/VIDAL-RICARDO-AP1

Para esta entrega se preparó la carpeta SITRAC-MET-AP4 con el código fuente, pom.xml, scripts SQL, diagramas, evidencias, README y documentación. La organización permite que el docente revise tanto el informe como los artefactos técnicos utilizados para producirlo.

También se preparó un guion de aproximadamente tres minutos para la presentación en video. El relato prioriza el problema abordado, la evolución incremental del proyecto, la arquitectura MVC, la demostración de las estructuras Java, la persistencia MySQL y el cierre. El video debe mostrar brevemente el programa ejecutándose para complementar la explicación oral.

13. Conclusiones

La cuarta entrega permite consolidar SITRAC-MET como un proyecto integrador y no como un conjunto aislado de ejercicios. La solución conserva el análisis y el diseño desarrollados previamente, y los transforma en una arquitectura Java organizada mediante MVC. La lógica orientada a objetos continúa representando el dominio, mientras que la persistencia se desacopla mediante interfaces y DAO.

La utilización conjunta de arreglos y ArrayList responde a necesidades distintas del programa; las excepciones controlan tanto reglas del negocio como errores técnicos; y el archivo de auditoría agrega una práctica complementaria sin reemplazar MySQL. De esta forma, el prototipo integra análisis, diseño, programación orientada a objetos, estructuras de datos, persistencia y comunicación del proyecto dentro de una misma solución evolutiva.

Como continuación futura podrían incorporarse una interfaz gráfica, autenticación por roles y notificaciones automáticas de calibraciones próximas a vencer. Estas mejoras no forman parte del alcance de la actividad actual, pero la separación de responsabilidades aplicada facilita su incorporación sin rediseñar completamente el sistema.

14. Referencias bibliográficas

Debrauwer, L. (2013). Patrones de diseño en Java. Ediciones ENI.

Deitel, P. J., & Deitel, H. M. (2016). Cómo programar en Java. Pearson.

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas. Pearson Educación.

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico. McGraw-Hill.

Universidad Siglo 21. (2026). Módulo 4: Patrones de diseño y JDBC [Material de cátedra].